

What is OOP? Explain its features.

* Define OOP :

Object Oriented Programming (OOP) is a programming paradigm that uses classes and objects to design and develop software.

It organizes a program around real-world entities rather than functions and logic.

* What is OOP ?

OOP stands for Object Oriented Programming.

It is a way of programming in which a program is organized using objects and classes.

An object is a real-world entity that has properties (data) and behaviors (functions).

* Advantages of OOP:

- ① Code Reusability – Code can be reused using inheritance.
- ② Data Security – Encapsulation protects data from unauthorized access.
- ③ Easy Maintenance – Code is well organized and easy to modify.
- ④ Flexibility – Polymorphism allows one interface with multiple implementations.
- ⑤ Modularity – Large programs are divided into smaller classes.
- ⑥ Real-world Modeling – It represents real-world scenarios in a better way.

* Real-world Examples of OOP:

① Car

- Class : Car
- Objects : BMW, Audi, Tesla
- Properties : Color, Model, Speed
- Methods : Start(), Stop(), Accelerate()

② Student

- Class : Student
- Objects : Ganesh, Rahul
- Properties : Name, Roll No., Marks
- Methods : Study(), AttendClass(), GiveExam()

③ Mobile Phone

- Class : Mobile
- Objects : Samsung, iPhone
- Properties : Brand, RAM, Storage
- Methods : Call(), Message(), Camera()

* Features of OOP :

- ① Class : A class is a blueprint or template used to create objects.
- ② Object : An object is an instance of a class. It represents a real-world entity.
- ③ Encapsulation : Encapsulation is the process of wrapping data and methods into a single unit (class) and restricting direct access to some of the object's components.
- ④ Abstraction : Abstraction means hiding the internal implementation details and showing only the essential features.
- ⑤ Inheritance : Inheritance allows one class to acquire the properties and methods of another class.
- ⑥ Polymorphism : Polymorphism means "one interface, multiple forms". It allows the same method to perform different tasks.

--- OOP makes code simple, reusable, secure and easy to maintain. ---

What is the difference between Class and Object?

* Definition :

① Class :

A class is a blueprint or template used to create objects. It defines the properties (attributes) and behaviors (methods) that the objects will have.

② Object :

An object is an instance of a class. It represents a real-world entity and has its own identity, state and behavior.

* Syntax :

① Class Syntax :

```
class ClassName {
    // attributes
    dataType attribute;
    // methods
    returnType methodName() {
        // code
    }
}
```

② Object Syntax :

```
ClassName objectName = new ClassName();

// Example
Student s1 = new Student();
```

* Examples :

① Class Example :

```
class Car {
    String brand;
    int speed;
    void start() {
        System.out.println("Car Started");
    }
}
```

② Object Example :

```
Car c1 = new Car();
c1.brand = "Toyota";
c1.speed = 120;
c1.start();
// Here, c1 is an object of Car class.
```

* Comparison Table :

Basis	Class	Object
1. Definition	A class is a blueprint or template.	An object is an instance of a class.
2. Creation	Created using 'class' keyword.	Created using 'new' keyword.
3. Memory	Does not occupy memory.	Occupies memory.
4. Existence	Logical entity.	Physical entity.
5. Example	class Student { ... }	Student s1 = new Student();
6. Purpose	Defines properties and behaviors.	Uses properties and behaviors.
7. Number	One class can have multiple objects.	Each object is an instance of a class.

* Summary :

Class is the blueprint, Object is the real entity created from the blueprint.

Explain Encapsulation with an example.

* Definition :

Encapsulation is the process of wrapping data (variables) and methods into a single unit (class). It restricts direct access to some of the object's components and can prevent accidental modification of data.

* Private Variables :

In encapsulation, the variables of a class are declared as private using the keyword 'private' so that they cannot be accessed directly from outside the class.

* Getter & Setter Methods :

To access and update the value of private variables, we use public methods called Getter (get) and Setter (set) methods.

- Getter method - returns the value of a private variable.
- Setter method - sets or updates the value of a private variable.

* Advantages :

- ① Data hiding - protects data from direct access.
- ② Data security - prevents unauthorized modification.
- ③ Better control over data.
- ④ Code reusability and modularity.
- ⑤ Easy maintenance and debugging.

* Example Program (Java) :

```
class Student {  
    // private variables  
    private String name;  
    private int age;  
  
    // Setter methods  
    public void setName(String name) {  
        this.name = name;  
    }  
    public void setAge(int age) {  
        if (age > 0) {  
            this.age = age;  
        } else {  
            System.out.println("Invalid age");  
        }  
    }  
  
    // Getter methods  
    public String getName() {  
        return name;  
    }  
    public int getAge() {  
        return age;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
  
        s1.setName("Ganesh");  
        s1.setAge(21);  
  
        System.out.println("Name : " + s1.getName());  
        System.out.println("Age : " + s1.getAge());  
  
        // trying invalid age  
        s1.setAge(-5);  
    }  
}
```

Output :

```
Name : Ganesh  
Age : 21  
Invalid age
```

- ## * Summary :
- Encapsulation binds data and methods together and restricts direct access to data using private variables and accessing them through getter and setter methods.

Explain Abstraction with an example.

* Definition :

Abstraction is the process of hiding the internal implementation details and showing only the essential features of an object to the user. It helps in reducing complexity and increasing security.

* Interface :

- An interface is a reference type in Java.
- It contains only abstract methods (by default public and abstract) and constant variables.
- It cannot have method body (before Java 8).
- A class implements an interface using the keyword 'implements' and must provide the implementation for all methods.

Example :

```
interface Vehicle {
    void start();    // abstract method
    void stop();    // abstract method
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car Started");
    }
    public void stop() {
        System.out.println("Car Stopped");
    }
}

public class Test {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();
        v.stop();
    }
}
```

* Abstract Class :

- An abstract class is a class that cannot be instantiated.
- It may contain abstract methods (without body) and concrete methods (with body).
- Abstract methods must be implemented by its subclasses.

Example :

```
abstract class Shape {
    abstract void draw(); // abstract method
    void color() {
        System.out.println("Color is red"); // concrete method
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

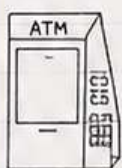
// Usage
public class Main {
    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw();
        s.color();
    }
}
```

* Advantages of Abstraction :

- ① Hides internal implementation details and shows only essential features.
- ② Reduces complexity and improves security.
- ③ Helps in achieving loose coupling.
- ④ Easy to maintain and modify.
- ⑤ Supports reusability and scalability of code.

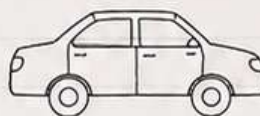
* Real-life Examples :

① ATM Example :



- We know how to use an ATM.
- We insert card, enter PIN, select option and get cash.
- But we don't know the internal working like how it verifies PIN, how cash is dispensed, etc.
- This is Abstraction - showing only important features to the user.

② Car Example :



- We drive the car using steering, accelerator and brake.
- We don't need to know how engine works internally.
- The internal mechanics are hidden from us.
- This is Abstraction.

* Summary : Abstraction helps to focus on what an object does instead of how it does it.

Explain Inheritance and its Types.

* Definition :

Inheritance is an OOPs concept that allows a new class (derived class / child class) to acquire the properties and behaviors (methods) of an existing class (base class / parent class). It promotes code reusability and establishes a relationship between classes.

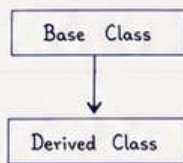
* Advantages :

- ① Code reusability.
- ② Reduces redundancy and improves maintainability.
- ③ Extensibility – new features can be added easily.
- ④ Establishes a relationship between classes.
- ⑤ Improves modularity and scalability of code.

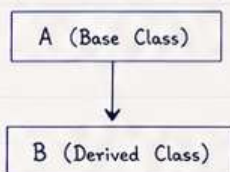
* Types of Inheritance :

① Single Inheritance :

A derived class inherits from a single base class.

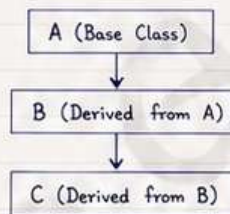


Example : class B extends A

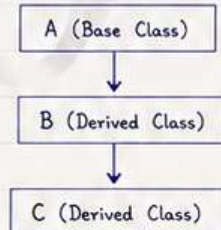


② Multilevel Inheritance :

A derived class inherits from another derived class.

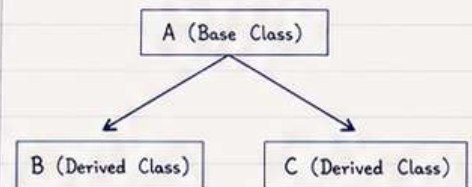


Example : class C extends B

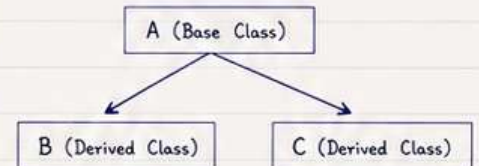


③ Hierarchical Inheritance :

Multiple derived classes inherit from a single base class.

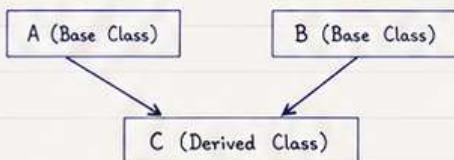


Example : class B extends A
class C extends A



④ Multiple Inheritance :

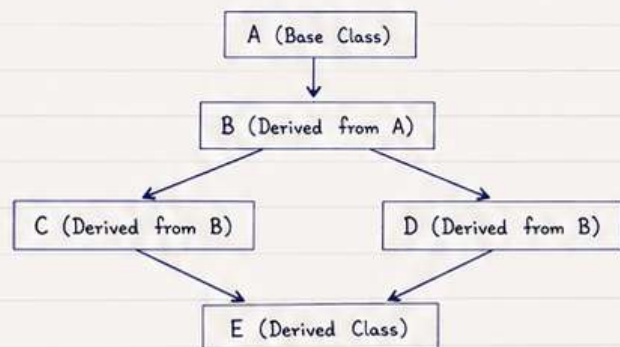
A class inherits from more than one base class. (Supported in C++ through classes, in Java using interfaces.)



Example : class C : public A, public B (C++)
interface A { }
interface B { }
class C implements A, B { } (Java)

⑤ Hybrid Inheritance :

A combination of two or more types of inheritance.



Example : Combination of Multilevel and Hierarchical.

* Note : In Java, multiple inheritance is not supported with classes to avoid ambiguity. It is supported using interfaces.

Explain Polymorphism with Examples.

* Definition :

Polymorphism means "one interface, multiple forms".

It allows the same method or function name to behave differently in different situations.

It increases the flexibility and reusability of code.

① Compile-time Polymorphism (Static Polymorphism)

It is achieved by method overloading.
The decision is taken by the compiler.

A) Method Overloading :

In the same class, multiple methods having the same name but different parameters (number, type or order).

Example :

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    float add(float a, float b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

class Test {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20)); // 30
        System.out.println(c.add(10.5f, 20.5f)); // 31.0
        System.out.println(c.add(10, 20, 30)); // 60
    }
}
```

② Run-time Polymorphism (Dynamic Polymorphism)

It is achieved by method overriding.
The decision is taken at runtime.

B) Method Overriding :

In the child class, a method is redefined with the same name, same parameters and same return type as in the parent class.

Example :

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Animal();
        a.sound(); // Animal makes sound

        a = new Dog();
        a.sound(); // Dog barks
    }
}
```

* Differences :

Aspect	Compile-time Polymorphism (Method Overloading)	Run-time Polymorphism (Method Overriding)
1. Binding	Early binding	Late binding
2. Decision	Taken by compiler	Taken at runtime
3. Achieved by	Method overloading	Method overriding
4. Inheritance	Not required	Required
5. Method Signature	Must be different	Must be same
6. Return Type	Can be different	Must be same (or covariant)
7. Example	<u>add</u> (int a, int b) <u>add</u> (int a, int b, int c)	<u>void</u> sound() in Animal <u>void</u> sound() in Dog

* Note : Polymorphism makes programs more flexible and extensible.
It allows new classes to be added with new behavior without changing the existing code.

Difference between Method Overloading and Method Overriding.

Method Overloading and Method Overriding are two different ways to achieve polymorphism in OOPs.

Topic	Method Overloading	Method Overriding
① Definition	Method Overloading is a feature in the same class where multiple methods have the same name but different parameters.	Method Overriding occurs in two classes (parent and child) where the child class provides a specific implementation of a method already defined in the parent class.
② Parameters	Must be different in number, type or order.	Must be same (same number, type and order).
③ Return type	Can be different.	Must be same or covariant (in case of objects).
④ Inheritance	Not required. It occurs in the same class.	Required. It occurs in parent and child classes.
⑤ Compile time vs Runtime	Resolved at compile time. (Early Binding / Static Binding)	Resolved at runtime. (Late Binding / Dynamic Binding)

Constructor in OOP.

* Definition :

A constructor is a special member function of a class that is automatically called when an object of the class is created. It has the same name as the class and does not have a return type (not even void).

* Purpose :

- To initialize objects.
- To set initial values to the data members.
- To allocate memory resources if needed.
- It is called automatically when an object is created.

* Types of Constructors :

- ① Default Constructor
- ② Parameterized Constructor
- ③ Copy Constructor (In C++)

① Default Constructor :

A constructor which has no parameters. If we don't create any constructor, compiler provides a default constructor automatically.

Example (C++) :

```
class Student {
    int roll;
public:
    Student() { // default constructor
        roll = 0;
    }
};
```

② Parameterized Constructor :

A constructor which takes parameters. It is used to initialize the object with specific values.

Example (C++) :

```
class Student {
    int roll;
    string name;
public:
    Student(int r, string n) { // parameterized
        roll = r; // constructor
        name = n;
    }
};
```

③ Copy Constructor (C++) :

A constructor which creates a new object as a copy of an existing object.

Example (C++) :

```
class Student {
    int roll;
    string name;
public:
    Student(Student &s) { // copy constructor
        roll = s.roll;
        name = s.name;
    }
};
```

* Constructor vs Method :

Aspect	Constructor	Method
1. Name	Same as class name	Can be any valid name
2. Return Type	No return type (not even void)	Must have a return type or void
3. Call	Called automatically when object is created	Called manually
4. Purpose	To initialize the object	To perform operations
5. Inheritance	Constructor is not inherited	Methods are inherited
6. Overloading	Can be overloaded	Can be overloaded
7. Default Provided	Compiler provides default constructor if no constructor is defined.	No default method is provided by compiler.

* Summary : Constructors are special methods used to initialize objects. They are called automatically and can be of different types depending on the requirements.

Explain Access Modifiers.

* Introduction :

Access modifiers in OOPs are used to set the accessibility of classes, variables, methods and constructors. They help in data hiding and information security.

* Types of Access Modifiers :

① Public :

- Members declared as public are accessible from anywhere in the program.
- There is no restriction on access.

② Private :

- Members declared as private are accessible only within the same class.
- They are not accessible from outside the class.

③ Protected :

- Members declared as protected are accessible within the same class, same package and in derived classes (even if they are in different packages).
- It provides more visibility than private but less than public.

④ Default (Java) [No Modifier] :

- If we do not specify any access modifier, it is treated as default.
- In Java, default members are accessible only within the same package.
- They are not accessible outside the package.

Notes :

- More restrictive → private
- Less restrictive → public

Order of Accessibility :

private < default < protected < public

* Comparison Table :

Access Modifier	Accessibility				Explanation
	Same Class	Same Package (Subclass or not)	Different Package (Subclass)	Different Package (Non-Subclass)	
1. Public	✓	✓	✓	✓	Accessible from everywhere.
2. Protected	✓	✓	✓	X	Accessible within same class, same package and in derived classes (even in different packages).
3. Default (No Modifier)	✓	✓	X	X	Accessible within same package only.
4. Private	✓	X	X	X	Accessible within the same class only.

* Summary : Access modifiers control the visibility of members in a program. They help in data hiding, code reusability and security.

What are the Four Pillars of OOP ?

* In Object Oriented Programming (OOP), there are four fundamental principles called the four pillars of OOP.

The Four Pillars of OOP

- ① Encapsulation
- ② Abstraction
- ③ Inheritance
- ④ Polymorphism

① Encapsulation

- Encapsulation is the concept of wrapping data (variables) and code (methods) together as a single unit.
- It binds the data and the methods that operate on the data into one unit and also restricts direct access to some of the object's components.
- It is achieved by using access modifiers (private, public, protected).

Real-life example :

A capsule in medicine. The medicine is wrapped inside a capsule and cannot be seen from outside. We can only use it, not see or change its internal chemical composition.

② Abstraction

- Abstraction means showing only the essential features and hiding the unnecessary details.
- It reduces complexity and allows us to focus on what an object does instead of how it does it.
- It is achieved using abstract classes and interfaces.

Real-life example :

When we drive a car, we only know how to drive (what it does), but we don't know how the engine, gearbox, or brakes work (how it does).

③ Inheritance

- Inheritance is the mechanism in which one class acquires the properties and behaviors of another class.
- It promotes code reusability and establishes a relationship between classes.
- The class that inherits is called subclass (child) and the class whose properties are inherited is called superclass (parent).

Real-life example :

A child inherits traits from parents like eye color, height, etc.

④ Polymorphism

- Polymorphism means "one interface, multiple forms".
- It allows the same method or function name to behave differently in different situations.
- It is achieved by method overloading (compile-time) and method overriding (run-time).

Real-life example :

A person can act as a teacher in a classroom, as a driver on the road, and as a musician on stage. The same person shows different behavior in different situations.

Summary : The four pillars of OOP (Encapsulation, Abstraction, Inheritance and Polymorphism) work together to make the software more secure, flexible, reusable and easy to maintain.